

Architectural and Algorithmic Roadmap for High-Fidelity Satellite Imagery

Hello Aditya. Architecting a system to handle geospatial reconstruction requires strict modularity, something that aligns perfectly with your systemic design background. Here is your detailed, step-by-step logical roadmap to construct this super-resolution pipeline, tailored specifically for deployment within your Antigravity IDE workspace.

Stage 1: Minimal Working SR (Swin2SR Inference Baseline)

File Structure

- /src/model_swin2sr.py
- /notebooks/stage1_inference.ipynb

Logic Flow

The initial objective is to establish structural fidelity using a Swin2SR Vision Transformer. To bypass thousands of hours of baseline training, the architecture dictates loading pre-trained weights from the Solafune Team N repository. This guarantees immediate geometric stability. The inference sequence involves loading a single Sentinel-2 low-resolution tensor, feeding it through the transformer backbone in evaluation mode without gradient tracking, and retrieving the mathematical baseline output.

Pseudocode

```
Python
import torch
from swin2sr import Swin2SR

def initialize_swin2sr_model(weight_path):
    model = Swin2SR(
        embed_dim=180,
        depths=[6, 6, 6, 6, 6, 6],
        num_heads=[6, 6, 6, 6, 6, 6],
        window_size=8,
        upscale=4,
    )
    state_dict = torch.load(weight_path)
    model.load_state_dict(state_dict)
    model.eval()
    return model
```

```
def run_inference(model, input_tensor):
    with torch.no_grad():
        output_tensor = model(input_tensor)
    return output_tensor
```

Stage 2: Proper Training Pipeline

File Structure

- /src/data_loader.py
- /src/metrics.py
- /src/utils_memory.py

Logic Flow

The pipeline must dynamically stream image patches directly into PyTorch memory buffers via the Google Earth Engine API to bypass local storage bottlenecks. For radiometric normalization, dividing the raw 16-bit array by 65,535 will critically crush the high-frequency urban gradients you need. Instead, the logic establishes a hard radiometric ceiling of 3000.0. Any pixel exceeding this ceiling is mathematically clamped to 3000.0, divided by the same value, and cast to a 32-bit float. Extraction utilizes strided window logic to pull manageable, overlapping blocks of data to avoid out-of-memory crashes.

Pseudocode

```
Python
import numpy as np

def normalize_sentinel_radiometry(raw_array):
    ceiling = 3000.0
    clamped_array = np.clip(raw_array, 0.0, ceiling)
    normalized_array = clamped_array / ceiling
    return normalized_array.astype(np.float32)

def extract_strided_windows(master_array, tile_size, overlap):
    stride = tile_size - overlap
    return np.lib.stride_tricks.as_strided(
        master_array,
        shape=(num_tiles_y, num_tiles_x, tile_size, tile_size),
        strides=(stride_y, stride_x, item_size_y, item_size_x),
    )
```

Stage 3: Fine-Tuning

File Structure

- /src/training_loop.py
- /configs/stage3_hyperparams.yaml

Logic Flow

The architecture is now fine-tuned on the WorldStrat dataset, mapping Airbus SPOT 6/7 high-resolution imagery against Sentinel-2 inputs across diverse geographic environments. The training loop calculates the loss and backpropagates errors to update the network weights. Optimization utilizes L1 loss rather than L2 loss to preserve the sharpness of median estimations and prevent organic blurring.

Pseudocode

```
Python
import torch
import torch.nn.functional as F

def execute_training_step(model, optimizer, lr_tensor, hr_tensor):
    optimizer.zero_grad()
    sr_prediction = model(lr_tensor)
    pixel_loss = F.l1_loss(sr_prediction, hr_tensor)
    pixel_loss.backward()
    optimizer.step()
    return pixel_loss.item()
```

Stage 4: Hallucination Guardrails

File Structure

- /src/loss_functions.py

Logic Flow

To guarantee geographical fidelity and explicitly penalize the fabrication of nonexistent geospatial features, strict mathematical guardrails are instituted. Cycle Consistency Loss forces the generated super-resolved output back to low-resolution dimensions using a degradation operator, mathematically defined as:

$$L_{\text{cycle}} = ||\text{Degrade}(SR) - LR||_1$$

A fidelity check is computed using the L1 norm. To maintain rigid structural geometries, a

Gradient Profile Loss minimizes the difference between the first-order spatial derivatives of the synthetic and ground truth images.

Pseudocode

```
Python
import torch
import torch.nn.functional as nn

def downsampling_consistency_loss(sr_tensor, lr_tensor, scale_factor):
    degraded_sr = F.interpolate(
        sr_tensor,
        scale_factor=1.0 / scale_factor,
        mode='bicubic',
        align_corners=False
    )
    return F.L1_loss(degraded_sr, lr_tensor)

def gradient_profile_loss(sr_tensor, hr_tensor):
    sr_grad_x = torch.abs(sr_tensor[:, :, :, 1] - sr_tensor[:, :, :, 0])
    sr_grad_y = torch.abs(sr_tensor[:, :, 1, :] - sr_tensor[:, :, 0, :])
    hr_grad_x = torch.abs(hr_tensor[:, :, :, 1] - hr_tensor[:, :, :, 0])
    hr_grad_y = torch.abs(hr_tensor[:, :, 1, :] - hr_tensor[:, :, 0, :])

    loss_x = F.L1_loss(sr_grad_x, hr_grad_x)
    loss_y = F.L1_loss(sr_grad_y, hr_grad_y)
    return loss_x + loss_y
```

Stage 5: Perceptual Enhancement (The Hybrid Ensemble)

File Structure

- /src/model_ensemble.py
- /src/utils_blending.py

Logic Flow

Because a single neural network cannot simultaneously minimize mathematical distortion and maximize visual realism, you will construct a 3-Tier Hybrid Ensemble Architecture. Outputs from the fidelity engine (Swin2SR) and the generative perception engine (Real-ESRGAN or a lightweight diffusion model) are merged via an Adaptive Fusion Engine. A dynamic blending parameter dictates the ratio; dense vegetation highly weights the Transformer to suppress

hallucinations, while structured urban grids heavily weight the GAN/Diffusion model to maximize sharpness. Out-of-core memory tiles are stitched together using a 2D Gaussian Blending Algorithm functioning as a spatial confidence map to eliminate grid artifacts.

Pseudocode

```
Python
import torch
import numpy as np

def adaptive_ensemble_fusion(fidelity_tensor, perception_tensor, alpha_map):
    fused_tensor = (alpha_map * fidelity_tensor) + (1.0 - alpha_map) * perception_tensor
    return fused_tensor

def generate_gaussian_window(window_size):
    window_1d = np.hanning(window_size)
    window_2d = np.outer(window_1d, window_1d)
    return torch.from_numpy(window_2d).float()

def accumulate_blended_tile(master_canvas, weight_canvas, tile_prediction, gaussian_window, y,
x, size):
    master_canvas[y:y+size, x:x+size] += tile_prediction * gaussian_window
    weight_canvas[y:y+size, x:x+size] += gaussian_window
    return master_canvas, weight_canvas
```

Structured Missing Pieces Checklist

During Stage 1 & 2 (Inference & Data Pipeline)

- Missing Piece 4 (Baseline Code Structure): Establish your actual PyTorch boilerplate within Antigravity IDE. Focus heavily on writing clean dataset loaders and inference scripts before you attempt any distributed training architecture.
- Missing Piece 3 (Dataset Realities): Actively handle real-world messiness in your GEE pipeline. Write specific utility functions to deal with cloud contamination bitmasks, HR/LR misalignment, and differing seasonal color calibrations.
- Missing Piece 2 (Debugging - API): Implement retry logic and exponential backoffs to handle Google Earth Engine API throttling and usage limits as you stream concurrent patches.

During Stage 3 (Fine-Tuning)

- Missing Piece 1 (Training Recipes): This requires strict experimentation. Iterate to find

your exact Learning Rate schedule, warmup strategy, optimizer momentum, gradient clipping thresholds, and patch sampling ratios for urban vs. rural targets.

- Missing Piece 4 (Advanced Structure): Expand your codebase to include proper epoch checkpointing, persistent experiment tracking, and a robust, fault-tolerant training loop.

During Stage 4 (Hallucination Guardrails)

- Missing Piece 2 (Debugging - Mathematics): As you introduce custom constraints like cycle consistency, closely monitor your training loop for exploding attention loss, NaNs in your tensors, and spectral inconsistencies across the RGB bands.

During Stage 5 (Perceptual Enhancement)

- Missing Piece 5 (Empirical Decisions): Make final, experimental decisions on your model stack. Empirically test whether SwinIR, Swin2SR, HAT, diffusion refinement, or Real-ESRGAN provides the best balance of inference speed and perceptual quality for your specific fusion engine.
- Missing Piece 2 (Debugging - Generative Artifacts): As you integrate the GAN or Diffusion paths, actively debug checkerboard artifacts, GAN mode collapse, and tile seams, ensuring your Gaussian blending matrix is properly normalized at the edges.

Which specific file from Stage 1 would you like to begin structuring within your IDE first?